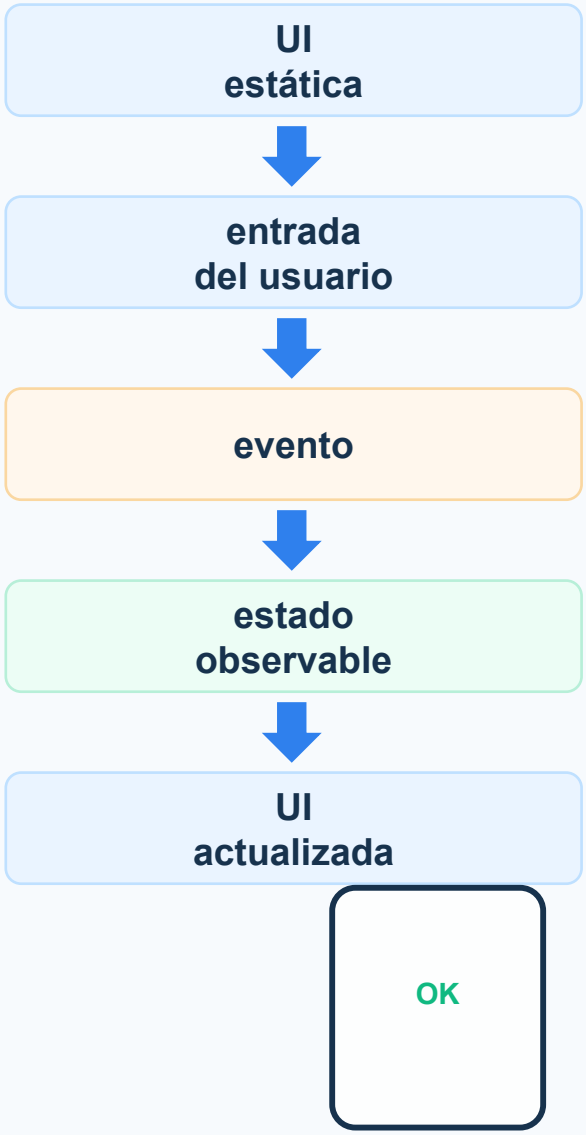
Kotlin aplicado, interacción y estado en Compose

introduce → valida → guarda → calcula → muestra → vacía



Kotlin que vamos a usar

Repaso operativo

```
val nombre = "Ane"
var intentos = 0

intentos += 1

val activo: Boolean = true
val precio: Double = 12.5

Text("Hola, $nombre")
```

Inferencia

```
val edad = 20
```

Kotlin deduce que edad es Int.

Precisión importante

```
val numeros = mutableListOf<Int>()
numeros.add(4)
```

val
referencia no reasignable



objeto mutable
contenido puede cambiar

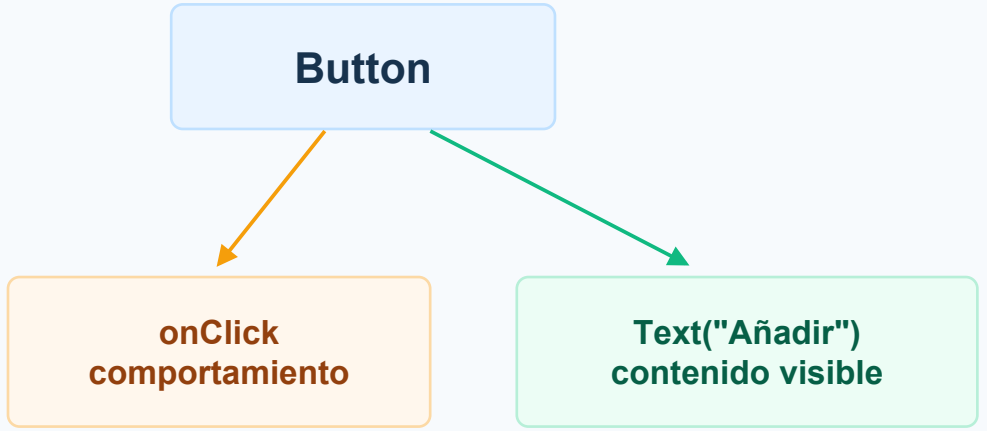
val → la referencia no se reasigna.
El objeto referenciado puede ser mutable.

Un evento: Button y onClick

Un botón tiene contenido visible y comportamiento.

Fragmento dentro de una función @Composable. Imports omitidos.

```
Button(
    onClick = {
        // qué hacer al pulsar
    }
) {
    Text("Añadir")
}
```




```
Button
├─ onClick → comportamiento
└─ Text    → contenido visible
```

onClick = { ... }

“cuando se pulse, ejecuta este bloque”

Una variable local no basta

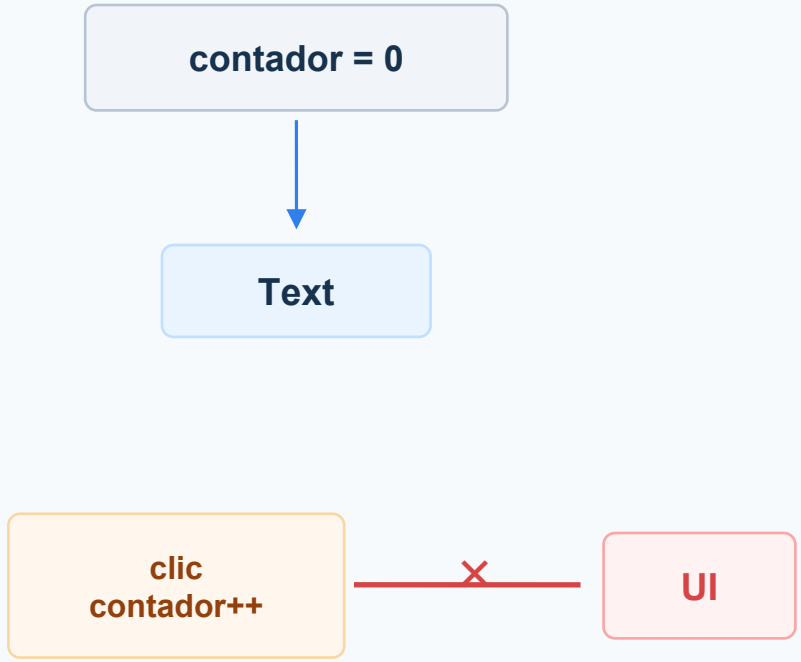
Este código compila, pero no modela correctamente estado de UI.

Imports omitidos.

```
@Composable
fun Contador() {
    var contador = 0

    Column {
        Text("Contador: $contador")

        Button(
            onClick = {
                contador++
            }
        ) {
            Text("Sumar")
        }
    }
}
```



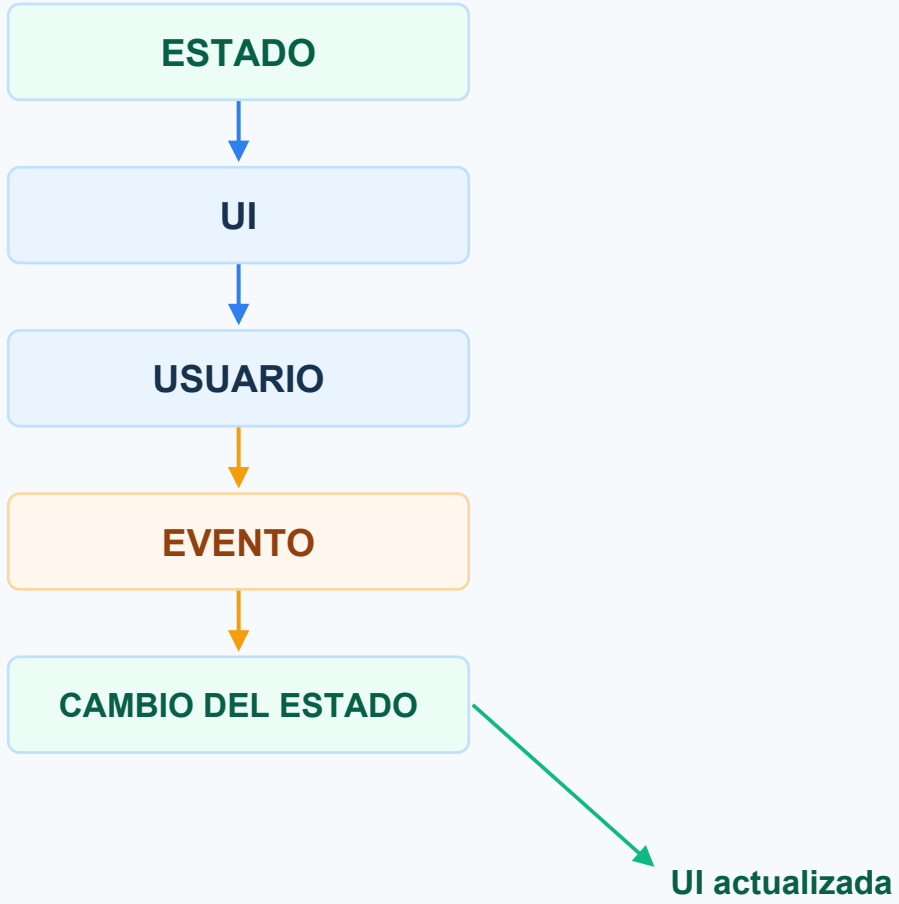
Problema

- no es estado observable para Compose
- una nueva ejecución de Contador() vuelve a crear contador = 0

El modelo central: estado y eventos

En Compose

estado leído por la UI
↓
cambia ese estado
↓
Compose puede recomponer
los lectores afectados
↓
la UI representa el nuevo valor



Recordatorio de la presentación anterior

Recomposición ≠ recrear la Activity
Recomposición ≠ volver a onCreate()

remember, mutableStateOf y by

Fragmento dentro de una función @Composable. Imports de runtime omitidos por legibilidad.

```
var contador by remember {
    mutableStateOf(0)
}
```

mutableStateOf(0)
crea estado mutable observable

remember { ... }
conserva entre recomposiciones mientras la entrada siga en composición

```
Text("Contador: $contador")

Button(
    onClick = {
        contador++
    }
) {
    Text("Sumar")
}
```

by
permite leer y escribir directamente contador

No confundir

remember ≠ persistencia

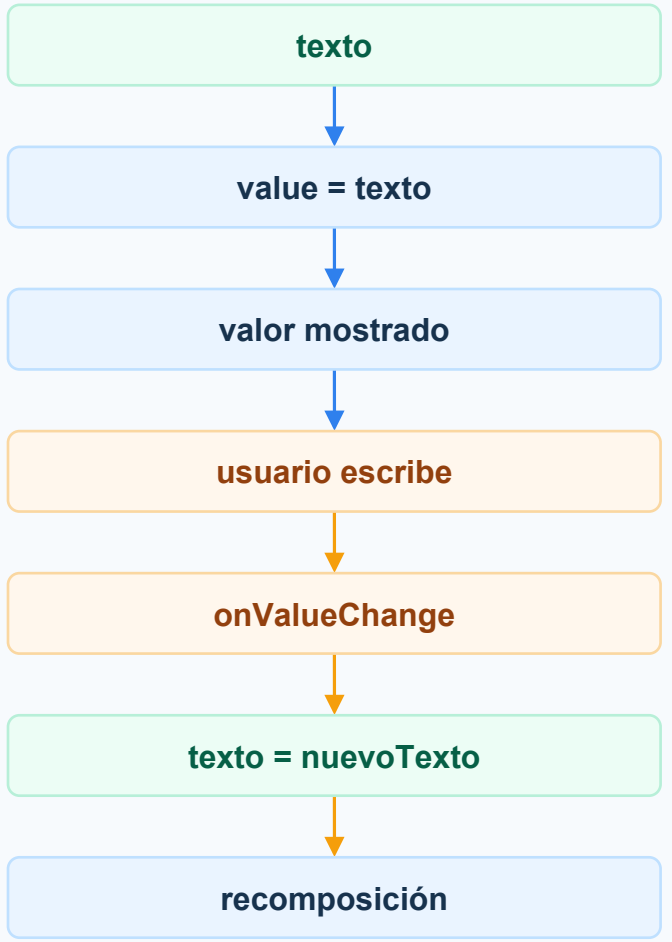
recomposición → conserva · salir/recrear composición → no garantizado · persistir datos → no

Un campo controlado por estado

Fragmento dentro de una función @Composable. Imports omitidos.

```
var texto by remember {
    mutableStateOf("")
}

OutlinedTextField(
    value = texto,
    onValueChange = { nuevoTexto ->
        texto = nuevoTexto
    },
    label = {
        Text("Número")
    },
    singleLine = true
)
```



value ← estado
onValueChange → nuevo estado

Del texto al dato: validar antes de actuar

Un campo contiene String, aunque escribamos cifras.

```
val numero: Int = texto.toInt()
```

Puede fallar si texto no representa un entero.

"25" ↓ 25

"hola" ↓ null

"" ↓ null

null ≠ ""

Conversión segura

```
val numero: Int? = texto.toIntOrNull()
```

Dentro de PrimeraPantalla:

```
...
var mensajeError by remember {
    mutableStateOf("")
}
...
Button(
    onClick = {
        val numero = texto.toIntOrNull()
        when {
            texto.isBlank() ->
                mensajeError = "Introduce un número"
            numero == null ->
                mensajeError = "Debe ser un entero"
            else ->
                mensajeError = ""
        }
    }
) {
    Text("Añadir")
}
if (mensajeError.isNotEmpty()) {
    Text(mensajeError)
}
...
```

Entrada incorrecta del usuario = situación esperable

Preferimos validarla. No usamos !! ni catch(Exception) como respuesta rutinaria.

Una lista mantiene varios datos

Solo lectura desde quien la recibe

List<Int>

Mutable cuando necesitamos modificarla

```
val numeros = mutableListOf<Int>()

numeros.add(4)
numeros.add(7)

for (numero in numeros) {
    println(numero)
}

numeros.remove(4)
numeros.clear()
```

Recordatorio

val numeros = ...

La referencia no cambia.
El contenido de la lista sí puede cambiar.

Operaciones que necesitamos

añadir · recorrer · eliminar · vaciar

Cuando la UI depende de una lista

Lista Kotlin ordinaria

```
val numeros = mutableListOf<Int>()
numeros.add(4)
```

Modificar una MutableList ordinaria no constituye, por sí solo, un cambio de estado observable por Compose.

MutableList mutación ✕ **estado observable de Compose**

Lista observable

```
val numeros = remember {
    mutableStateListOf<Int>()
}
```

```
numeros.add(4)
numeros.remove(4)
numeros.clear()
```

mutableStateListOf mutación → **cambio observable ↓ UI que la lee**

mutableStateListOf → crea una lista mutable observable por Compose.

Añadir y representar los números

Partimos del campo y la validación anteriores.

1. Estado nuevo

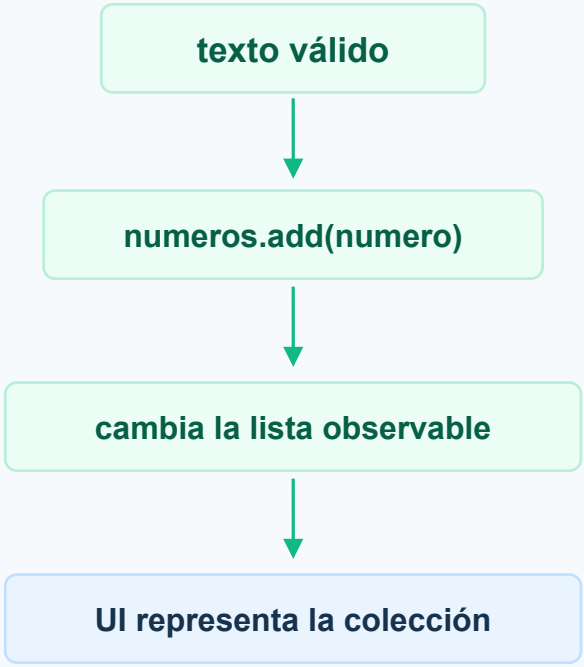
```
val numeros = remember {
    mutableStateListOf<Int>()
}
```

2. En la rama válida del when

```
else -> {
    numeros.add(numero)
    texto = ""
    mensajeError = ""
}
```

3. Para representar el estado

```
for (numero in numeros) {
    Text("• $numero")
}
```





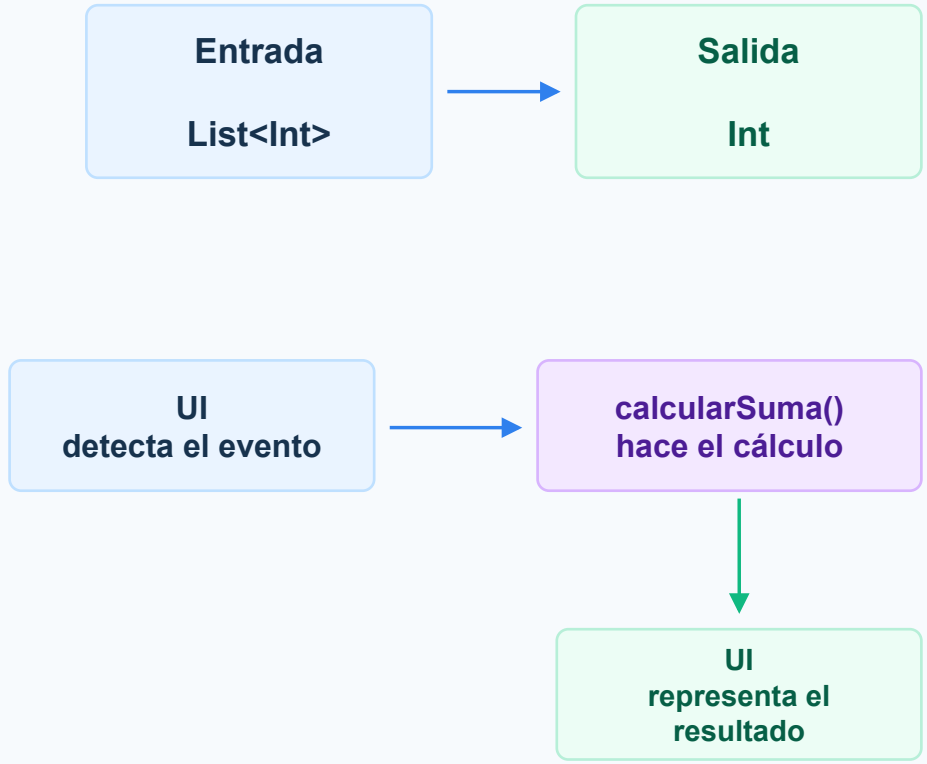
La función hace el cálculo

Ya tenemos datos. Ahora queremos procesarlos.

```
fun calcularSuma(numeros: List<Int>): Int {
    var suma = 0

    for (numero in numeros) {
        suma += numero
    }

    return suma
}
```



onClick ≠ lugar para meter todo el programa

Calcular, validar y mostrar resultado

Estado para un resultado que todavía puede no existir:

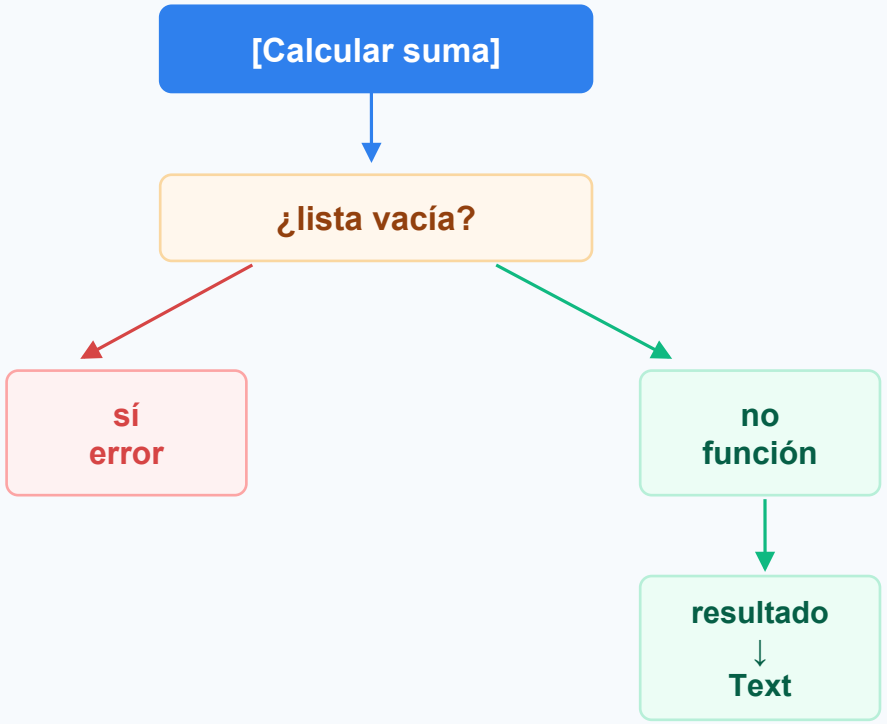
```
var resultado by remember {
    mutableStateOf<Int?>(null)
}
```

Representación:

```
if (resultado != null) {
    Text("Resultado: $resultado")
}
```

Botón:

```
Button(
    onClick = {
        if (numeros.isEmpty()) {
            mensajeError = "No hay números para calcular"
        } else {
            resultado = calcularSuma(numeros)
            mensajeError = ""
        }
    }
) {
    Text("Calcular suma")
}
```



Una clase modela información, no interfaz

Modelo con datos y comportamiento

```
class Persona(
    var nombre: String,
    var edad: Int
) {
    fun presentarse(): String {
        return "Hola, soy $nombre"
    }

    fun cumplirAnios(): String {
        edad++
        return "Ahora tengo $edad años"
    }
}
```

Crear y utilizar

```
val persona = Persona("Ane", 20)

val saludo = persona.presentarse()
val nuevaEdad = persona.cumplirAnios()
```

Resultado:

saludo → "Hola, soy Ane"
nuevaEdad → "Ahora tengo 21 años"

Modelo ≠ elemento visual

Cuando la finalidad principal es representar datos

```
data class Producto(
    val nombre: String,
    val precio: Double,
    val disponible: Boolean
)
```

Persona → objeto → datos + comportamiento

Vaciar significa recuperar un estado coherente


```
Button(
    onClick = {
        numeros.clear()
        texto = ""
        resultado = null
        mensajeError = ""
    }
) {
    Text("Vaciar")
}
```

Después de vaciar

numeros
→ []

texto
→ ""

resultado
→ null

mensajeError
→ ""

Código comprensible

texto
numeros
resultado
mensajeError
calcularSuma()

Calidad transversal

nombres claros · responsabilidades sencillas · sin duplicación evidente

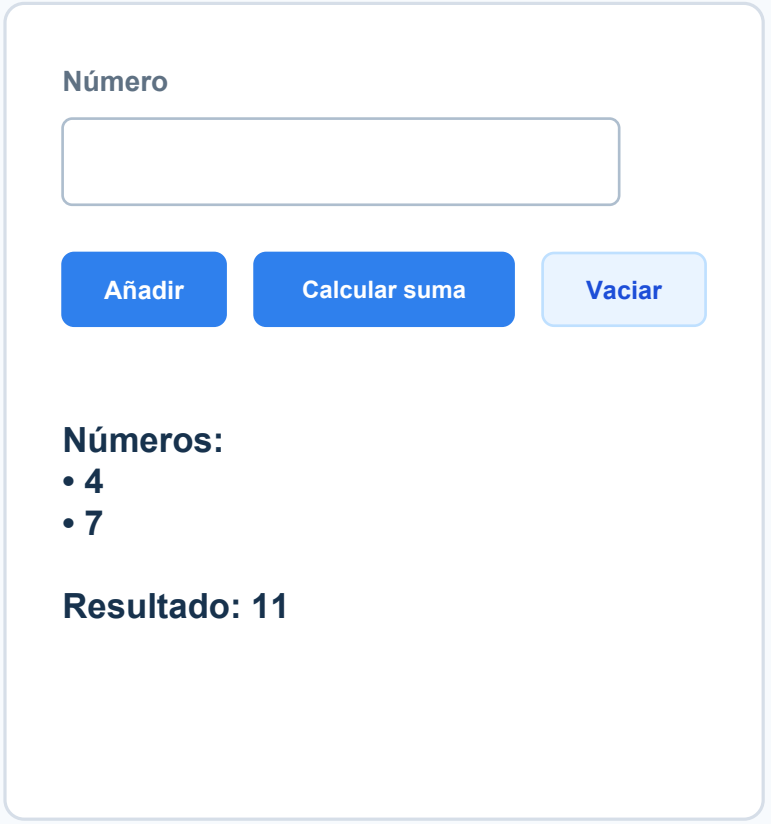
Microprácticа integradora

Partimos del ejemplo guiado que ya compila.

Completa o reconstruye las partes necesarias para que una sola pantalla permita:

- □ introducir un número
- □ detectar campo vacío
- □ detectar texto no numérico
- □ añadir un número válido
- □ mostrar los números almacenados
- □ calcular mediante función de suma
- □ rechazar el cálculo si la lista está vacía
- □ mostrar el resultado
- □ vaciar el estado de la pantalla

```
fun calcularSuma(numeros: List<Int>): Int
```



Pruebas a realizar en el emulador

1. **campo vacío**
2. **texto no numérico**
3. **dos números válidos**
4. **calcular**
5. **vaciar**

Cabe saber identificar:

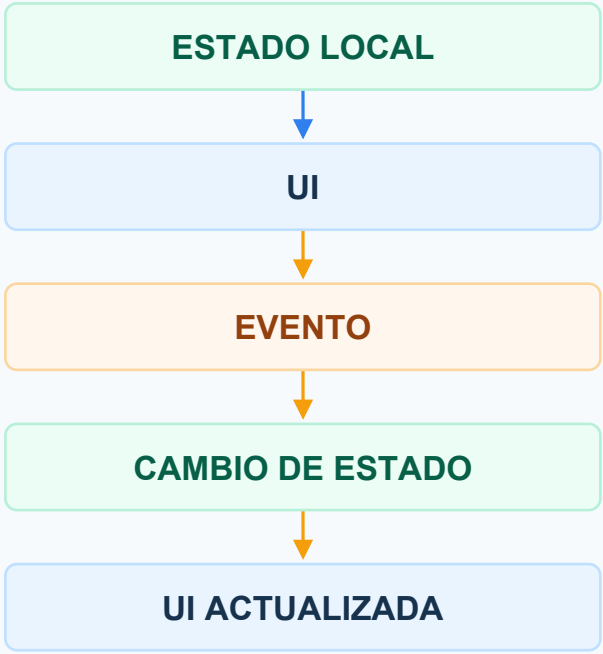
¿Dónde está el estado?
¿Qué evento lo modifica?
¿Qué función hace el cálculo?

Cierre: estado local resuelto

Ya podemos construir una pantalla que:

- recibe datos
- valida
- actualiza estado observable
- mantiene una colección
- ejecuta una función
- representa el resultado

Modelo de hoy



Pregunta para la siguiente presentación

¿Qué hacemos cuando el estado ya no pertenece solo a esta pantalla?